



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени
Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа по дисциплине «Операционные системы»

Тема Буферизованный и небуферизованный ввод-вывод

Студент Бакулин Д. Д.

Группа ИУ7-62Б

Преподаватели Рязанова Н. Ю.

Москва, 2026

1 Структуры

1.1 FILE и структура _IO_FILE

Листинг 1.1 — FILE /libio/bits/types/FILE.h

```
1 #ifndef __FILE_defined
2 #define __FILE_defined 1
3 struct _IO_FILE;
4 /* The opaque type of streams. This is the definition used
   elsewhere. */
5 typedef struct _IO_FILE FILE;
6 #endif
```

Листинг 1.2 — _IO_FILE /libio/bits/types/struct_FILE.h

```
1 struct _IO_FILE
2 {
3     int _flags; /* High-order word is _IO_MAGIC; rest is flags. */
4
5     /* The following pointers correspond to the C++ streambuf
       protocol. */
6     char *_IO_read_ptr; /* Current read pointer */
7     char *_IO_read_end; /* End of get area. */
8     char *_IO_read_base; /* Start of putback+get area. */
9     char *_IO_write_base; /* Start of put area. */
10    char *_IO_write_ptr; /* Current put pointer. */
11    char *_IO_write_end; /* End of put area. */
12    char *_IO_buf_base; /* Start of reserve area. */
13    char *_IO_buf_end; /* End of reserve area. */
14
15    /* The following fields are used to support backing up and undo.
       */
16    char *_IO_save_base; /* Pointer to start of non-current get area
       . */
17    char *_IO_backup_base; /* Pointer to first valid character of
       backup area */
18    char *_IO_save_end; /* Pointer to end of non-current get area.
       */
19
```

Продолжение листинга 1.2

```

20  struct _IO_marker *_markers;
21
22  struct _IO_FILE *_chain;
23
24  int _fileno;
25  int _flags2:24;
26  /* Fallback buffer to use when malloc fails to allocate one. */
27  char _short_backupbuf[1];
28  __off_t _old_offset; /* This used to be _offset but it's too
   small. */
29
30  /* 1+column number of pbase(); 0 is unknown. */
31  unsigned short _cur_column;
32  signed char _vtable_offset;
33  char _shortbuf[1];
34
35  _IO_lock_t *_lock;
36  #ifdef _IO_USE_OLD_IO_FILE
37 };
38
39 struct _IO_FILE_complete
40 {
41     struct _IO_FILE _file;
42     #endif
43     __off64_t _offset;
44     /* Wide character stream stuff. */
45     struct _IO_codecvt *_codecvt;
46     struct _IO_wide_data *_wide_data;
47     struct _IO_FILE *_freeres_list;
48     void *_freeres_buf;
49     struct _IO_FILE **_prevchain;
50     int _mode;
51     #if __WORDSIZE == 64
52     int _unused3;
53     #endif
54     __uint64_t _total_written;
55     #if __WORDSIZE == 32
56     int _unused3;
57     #endif

```

Продолжение листинга 1.2

```
58  /* Make sure we don't get into trouble again. */
59  char _unused2[12 * sizeof (int) - 5 * sizeof (void *)];
60 };
```

1.2 Структура stat

Листинг 1.3 — struct stat

```
1 struct stat {
2     dev_t      st_dev;      /* устройство */
3     ino_t      st_ino;      /* inode */
4     mode_t     st_mode;     /* режим доступа */
5     nlink_t    st_nlink;    /* количество жестких ссылок */
6     uid_t      st_uid;      /* идентификатор пользователя-владельца */
7     gid_t      st_gid;      /* идентификатор группы-владельца */
8     dev_t      st_rdev;     /* тип устройства (если это устройство)*/
9     off_t      st_size;     /* общий размер в байтах */
10    blksize_t   st_blksize;  /* размер блока ввода-вывода в файловой си
       стеме */
11    blkcnt_t    st_blocks;   /* количество выделенных 512-байтных блоко
       в */
12    union { time_t st_atime; struct timespec st_atim; }; /* время по
       следнего доступа */
13    union { time_t st_mtime; struct timespec st_mtim; }; /* время по
       следней модификации */
14    union { time_t st_ctime; struct timespec st_ctim; }; /* время по
       следнего изменения */
15 };
```

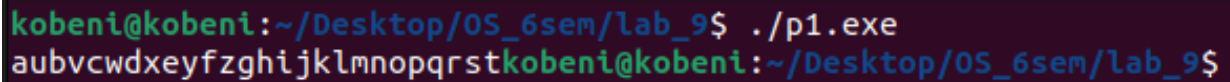
2 Программа №1

2.1 Код программы

Листинг 2.1 — Код первой программы

```
1 #include <stdio.h>
2 #include <fcntl.h>
3
4 int main()
5 {
6     int fd = open("alphabet.txt", O_RDONLY);
7
8     FILE *fs1 = fdopen(fd, "r");
9     char buff1[20];
10    setvbuf(fs1, buff1, _IOFBF, 20);
11
12    FILE *fs2 = fdopen(fd, "r");
13    char buff2[20];
14    setvbuf(fs2, buff2, _IOFBF, 20);
15
16    int flag1 = 1, flag2 = 2;
17    while(flag1 == 1 || flag2 == 1)
18    {
19        char c;
20        flag1 = fscanf(fs1, "%c", &c);
21        if (flag1 == 1)
22        {
23            fprintf(stdout, "%c", c);
24        }
25        flag2 = fscanf(fs2, "%c", &c);
26        if (flag2 == 1)
27        {
28            fprintf(stdout, "%c", c);
29        }
30    }
31    return 0;
32 }
```

2.2 Результат работы программы



```
kobenikobeni:~/Desktop/OS_6sem/lab_9$ ./p1.exe
aubvcwdxyfzghijklmnopqrstkobenikobeni:~/Desktop/OS_6sem/lab_9$
```

Рисунок 2.1 — Результат работы первой программы

2.3 Анализ работы программы

Системный вызов `open` открывает файл `alphabet.txt` в режиме «только для чтения», создаёт структуру `file`. Ссылка на эту структуру помещается в таблицу файловых дескрипторов процесса. Поскольку дескрипторы 0, 1 и 2 заняты стандартными потоками ввода-вывода, системный вызов возвращает наименьший свободный дескриптор (3), который присваивается переменной `fd`.

Библиотечной функции `fdopen` передается дескриптор `fd` и режим доступа «только для чтения». Функция возвращает указатель на инициализированную структуру `FILE`, где полю `_fileno` присваивается значение `fd`.

Функция `setvbuf` задаёт размер буфера 20 байт для `fs1` и `fs2`.

Функция `fscanf` для `fs1` сначала считывает 20 символов («a»—«t») из файла `"alphabet.txt"` в буфер, при этом значение поля `f_pos` структуры `file` увеличивается на 20. Затем из буфера извлекается первый символ («a») и записывается в переменную `s`, а указатель `_IO_read_ptr` в структуре `fs1` сдвигается на 1 байт. Вызов `fprintf` выводит этот символ в стандартный поток вывода.

Так как `fs1` и `fs2` ссылаются на один и тот же открытый файл, где значение `f_pos` равняется 20, то при вызове `fscanf` для `fs2` в буфер считываются оставшиеся 6 символов («u»—«z») из файла `"alphabet.txt"`. Первый символ из буфера `fs2` («u») записывается в переменную `s` и выводится в стандартный поток вывода.

В последующих итерациях цикла поочерёдно будут выводиться символы из буферов `fs1` и `fs2`. Когда будет достигнут конец буфера `fs2` (сравниваются указатели `_IO_read_ptr` и `_IO_read_end`), то программа продолжит выводить оставшиеся символы только из буфера `fs1` (вплоть до `t`), пока не будет достигнут конец буфера `fs1`.

2.4 Схема связи структур

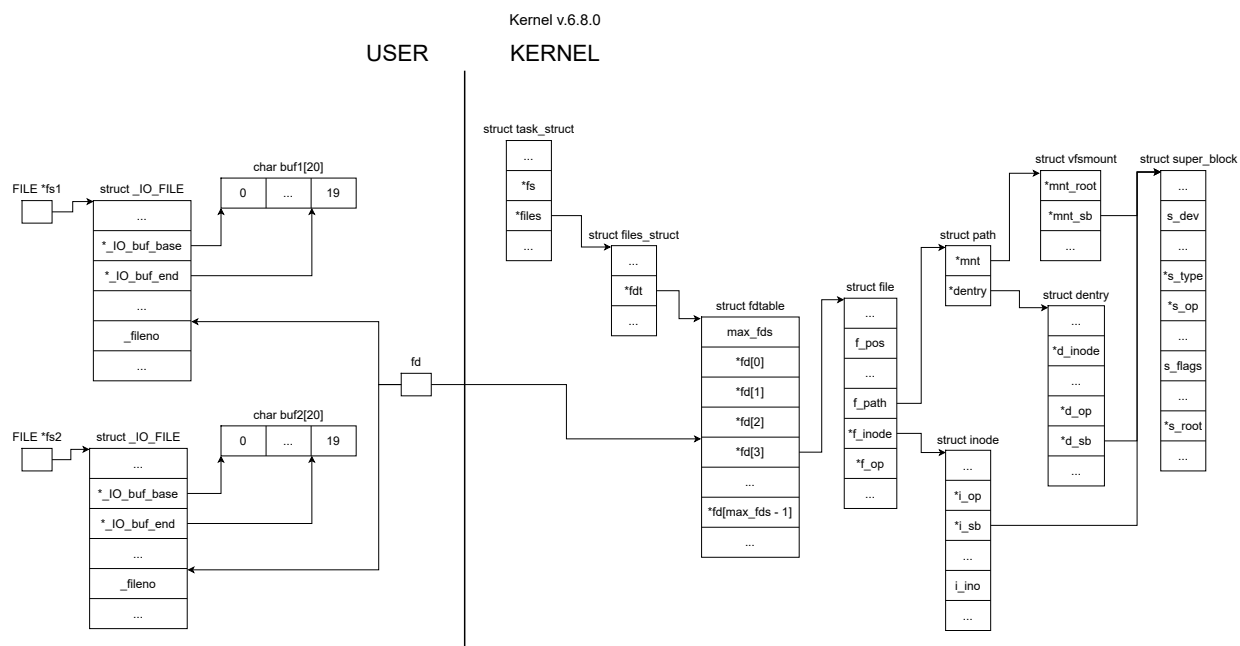


Рисунок 2.2 — Схема связи структур в первой программе

3 Программа №2

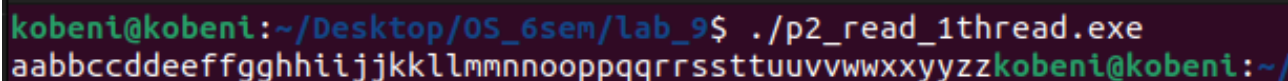
3.1 Однопоточный вариант программы

3.1.1 Код программы

Листинг 3.1 — Код однопоточной первой программы

```
1 #include <fcntl.h>
2 #include <unistd.h>
3
4 int main()
5 {
6     char c;
7     int fl1 = 1, fl2 = 1;
8
9     int fd1 = open("alphabet.txt", O_RDONLY);
10    int fd2 = open("alphabet.txt", O_RDONLY);
11
12    while((fl1 == 1) && (fl2 == 1))
13    {
14        if ((fl1 = read(fd1, &c, 1)) == 1)
15            write(1, &c, 1);
16        if (fl1 == 1 && ((fl2 = read(fd2, &c, 1)) == 1))
17            write(1, &c, 1);
18    }
19    return 0;
20 }
```

3.1.2 Результат работы программы



```
kobeni@kobeni:~/Desktop/OS_6sem/lab_9$ ./p2_read_1thread.exe
aabbccddeeffgghhiijjkkllmmnnoppqqrrssttuuvvwxxyyzzkobeni@kobeni:~/Desktop/OS_6sem/lab_9$
```

Рисунок 3.1 — Результат работы однопоточной второй программы

3.1.3 Анализ работы программы

Системный вызов `open` создаёт два дескриптора открытого файла в режиме «только для чтения» (`fd1`, `fd2`). Соответствующие дескрипторам структуры `file` ссылаются на один и

тот же inode файла «alphabet.txt». Ссылки на эти структуры помещаются в таблицу открытых процессом файлов под индексами 3 и 4, так как дескрипторы 0,1 и 2 заняты стандартными потоками.

Системный вызов read для дескриптора fd1 считывает один символ («а»), что приводит к увеличению на 1 значения поля f_pos в соответствующей структуре file. Считанный символ выводится в стандартный поток вывода.

Так как дескрипторам fd1 и fd2 соответствуют независимые структуры file, то для fd2 значение поля f_pos равно 0. Поэтому при вызове read для fd2 будет считан тот же самый первый символ («а»), который также выведется в стандартный поток вывода.

Поочерёдное чтение и вывод символов будут продолжаться в цикле, пока значения f_pos в обеих структурах file не достигнут конца файла (размера файла, хранящегося в поле i_size и равного 26 байтам).

Таким образом, в стандартный поток вывода каждый символ алфавита будет выведен строго последовательно дважды.

3.2 Вариант программы с 1 дополнительным потоком

3.2.1 Код программы

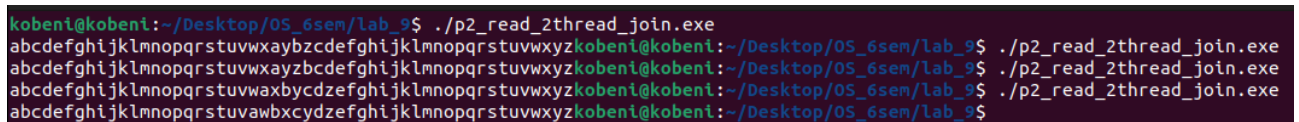
Листинг 3.2 — Код программы с 1 дополнительным потоком

```
1 #include <fcntl.h>
2 #include <pthread.h>
3 #include <unistd.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6
7 void *thread_func(void *arg)
8 {
9     char c;
10    int fl = 1;
11    int fd = *((int *)arg);
12
13    while (fl == 1)
14    {
15        if ((fl = read(fd, &c, 1)))
16            write(1, &c, 1);
17    }
18    return NULL;
19 }
```

Продолжение листинга 3.2

```
20
21 int main()
22 {
23     pthread_t t;
24     char c;
25     int fl = 1;
26
27     int fd1 = open("alphabet.txt", O_RDONLY);
28     int fd2 = open("alphabet.txt", O_RDONLY);
29
30     if (pthread_create(&t, NULL, thread_func, &fd1) != 0)
31     {
32         perror("pthread_create");
33         exit(1);
34     }
35
36     while (fl == 1)
37     {
38         if ((fl = read(fd2, &c, 1)) == 1)
39             write(1, &c, 1);
40     }
41     pthread_join(t, NULL);
42     return 0;
43 }
```

3.2.2 Результат работы программы



```
kobenikobeni:~/Desktop/OS_6sem/lab_9$ ./p2_read_2thread_join.exe
abcdefghijklmnopqrstuvwxyabcdefghijklmnopqrstuvwxykobenikobeni:~/Desktop/OS_6sem/lab_9$ ./p2_read_2thread_join.exe
abcdefghijklmnopqrstuvwxyabcdefghijklmnopqrstuvwxykobenikobeni:~/Desktop/OS_6sem/lab_9$ ./p2_read_2thread_join.exe
abcdefghijklmnopqrstuvwxyabcdefghijklmnopqrstuvwxykobenikobeni:~/Desktop/OS_6sem/lab_9$ ./p2_read_2thread_join.exe
abcdefghijklmnopqrstuvwxyabcdefghijklmnopqrstuvwxykobenikobeni:~/Desktop/OS_6sem/lab_9$
```

Рисунок 3.2 — Результат работы программы с 1 дополнительным потоком

3.2.3 Анализ работы программы

Вызов `pthread_create` создаёт дополнительный поток, которому в качестве аргумента передаётся указатель на дескриптор `fd1`. Поскольку на создание потока требуется время, главный поток может успеть прочитать и вывести либо весь алфавит, либо его часть.

После создания дополнительного потока чтение и вывод символов осуществляются обо-

ими потоками в зависящей от планировщика последовательности.

Когда главный поток завершит чтение по своему дескриптору, он выйдет из цикла и заблокируется в вызове `pthread_join`, ожидая завершения дополнительного потока. Дополнительный поток продолжит работу и выведет оставшуюся часть своих символов.

Таким образом, каждый символ файла будет напечатан дважды, однако точный порядок их вывода не определён и полностью зависит от того, как планировщик распределяет процессорное время между потоками.

3.3 Вариант программы с 2 дополнительными потоками

3.3.1 Код программы

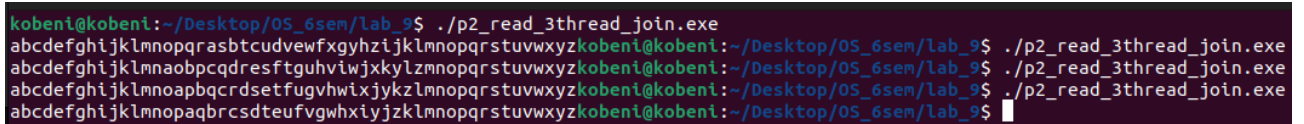
Листинг 3.3 — Код программы с 2 дополнительными потоками

```
1 #include <fcntl.h>
2 #include <pthread.h>
3 #include <unistd.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6
7 void *thread_func(void *arg)
8 {
9     char c;
10    int fl = 1;
11    int fd = *((int *)arg);
12
13    while (fl == 1)
14    {
15        if ((fl = read(fd, &c, 1)))
16            write(1, &c, 1);
17    }
18    return NULL;
19 }
20
21 int main()
22 {
23     pthread_t t1, t2;
24
25     int fd1 = open("alphabet.txt", O_RDONLY);
26     int fd2 = open("alphabet.txt", O_RDONLY);
```

Продолжение листинга 3.3

```
27
28  if (pthread_create(&t1, NULL, thread_func, &fd1) != 0)
29  {
30      perror("pthread_create");
31      exit(1);
32  }
33  if (pthread_create(&t2, NULL, thread_func, &fd2) != 0)
34  {
35      perror("pthread_create");
36      exit(1);
37  }
38
39  pthread_join(t1, NULL);
40  pthread_join(t2, NULL);
41
42  return 0;
43 }
```

3.3.2 Результат работы программы



```
kobenikobeni:~/Desktop/OS_6sem/lab_9$ ./p2_read_3thread_join.exe
abcdefghijklmnopqrstuvwxyzkobenikobeni:~/Desktop/OS_6sem/lab_9$ ./p2_read_3thread_join.exe
abcdefghijklmnopqrstuvwxyzkobenikobeni:~/Desktop/OS_6sem/lab_9$ ./p2_read_3thread_join.exe
abcdefghijklmnopqrstuvwxyzkobenikobeni:~/Desktop/OS_6sem/lab_9$ ./p2_read_3thread_join.exe
abcdefghijklmnopqrstuvwxyzkobenikobeni:~/Desktop/OS_6sem/lab_9$
```

Рисунок 3.3 — Результат работы программы с 2 дополнительными потоками

3.3.3 Анализ работы программы

Два вызова `pthread_create` создают два дополнительных потока. Каждому из них передается указатель на свой файловый дескриптор (первому — `fd1`, второму — `fd2`). Главный поток, создав рабочие потоки, сразу же блокируется в вызовах `pthread_join`, ожидая их завершения.

На создание потоков требуется время и их порядок выполнения зависит от планировщика, поэтому порядок вывода символов неизвестен.

Таким образом, каждый символ файла будет напечатан дважды, однако точный порядок их вывода не определен и полностью зависит от того, как планировщик распределяет процессорное время между дополнительными потоками.

3.4 Схема связи структур

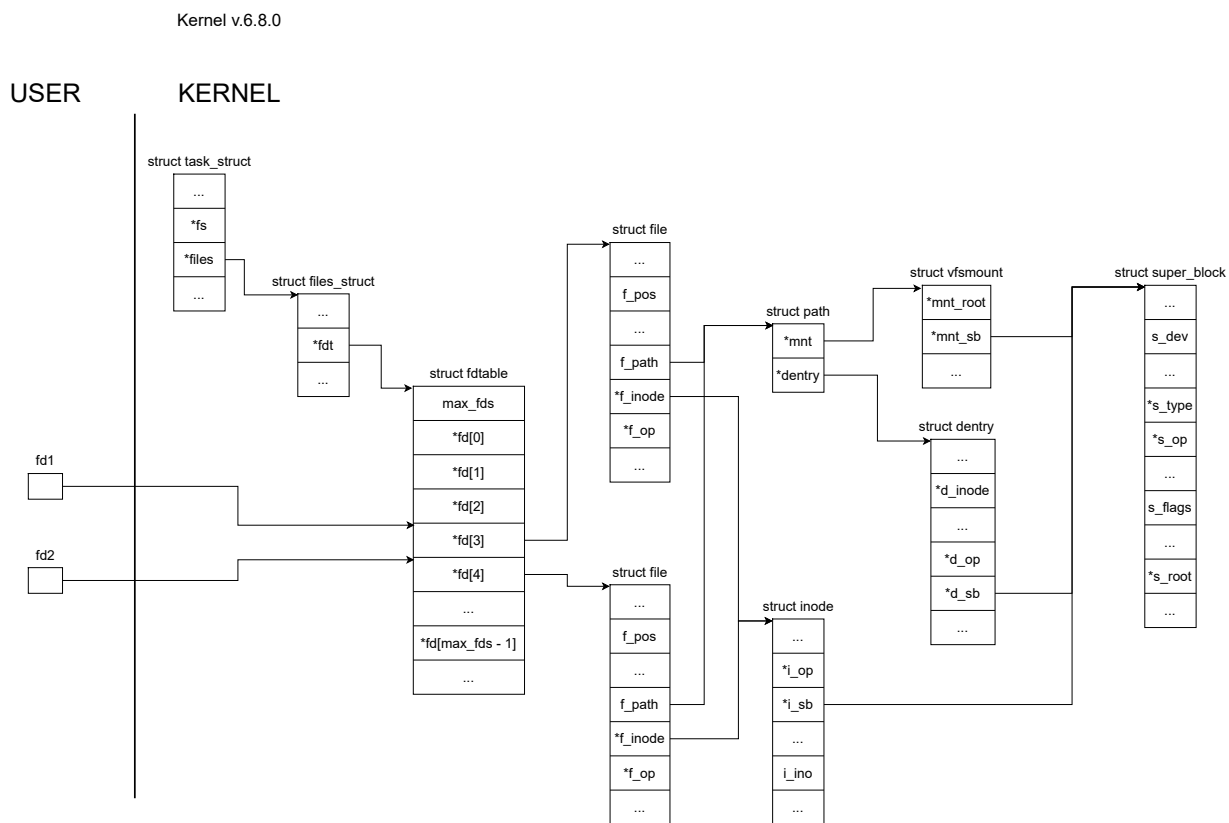


Рисунок 3.4 — Схема связи структур во второй программе

4 Программа №3

4.1 Вариант программы с open, write и close

4.1.1 Код программы

Листинг 4.1 — Код программы с open, write и close

```
1 #include <fcntl.h>
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <sys/stat.h>
5 #include <sys/types.h>
6
7 int main()
8 {
9     struct stat statbuf;
10    int fd1 = open("q.txt", O_RDWR);
11    int fd2 = open("q.txt", O_RDWR);
12    stat("q.txt", &statbuf);
13    printf("inode: %lu; size: %ld; blksize: %ld\n", statbuf.st_ino,
14           statbuf.st_size, statbuf.st_blksize);
15
16    for(char c = 'a'; c <= 'z'; c++)
17    {
18        if (c%2)
19            write(fd1, &c, 1);
20        else
21            write(fd2, &c, 1);
22        stat("q.txt", &statbuf);
23        printf("inode: %lu; size: %ld; blksize: %ld\n", statbuf.st_ino,
24               statbuf.st_size, statbuf.st_blksize);
25    }
26    close(fd1);
27    close(fd2);
28    return 0;
29 }
```

4.1.2 Результат работы программы

```
kobeni@kobeni:~/Desktop/OS_6sem/lab_9$ ./p3_write.exe
inode: 2359850;size: 0; blksize: 4096
inode: 2359850;size: 1; blksize: 4096
inode: 2359850;size: 1; blksize: 4096
inode: 2359850;size: 2; blksize: 4096
inode: 2359850;size: 2; blksize: 4096
inode: 2359850;size: 3; blksize: 4096
inode: 2359850;size: 3; blksize: 4096
inode: 2359850;size: 4; blksize: 4096
inode: 2359850;size: 4; blksize: 4096
inode: 2359850;size: 5; blksize: 4096
inode: 2359850;size: 5; blksize: 4096
inode: 2359850;size: 6; blksize: 4096
inode: 2359850;size: 6; blksize: 4096
inode: 2359850;size: 7; blksize: 4096
inode: 2359850;size: 7; blksize: 4096
inode: 2359850;size: 8; blksize: 4096
inode: 2359850;size: 8; blksize: 4096
inode: 2359850;size: 9; blksize: 4096
inode: 2359850;size: 9; blksize: 4096
inode: 2359850;size: 10; blksize: 4096
inode: 2359850;size: 10; blksize: 4096
inode: 2359850;size: 11; blksize: 4096
inode: 2359850;size: 11; blksize: 4096
inode: 2359850;size: 12; blksize: 4096
inode: 2359850;size: 12; blksize: 4096
inode: 2359850;size: 13; blksize: 4096
inode: 2359850;size: 13; blksize: 4096
kobeni@kobeni:~/Desktop/OS_6sem/lab_9$ cat q.txt
bdfhjlnprtvxz
kobeni@kobeni:~/Desktop/OS_6sem/lab_9$
```

Рисунок 4.1 — Результат работы программы с open, write и close

4.2 Анализ работы программы

Системный вызов `open` создаёт два дескриптора открытого файла в режиме «чтения и записи» (`fd1`, `fd2`). Соответствующие дескрипторам структуры `file` ссылаются на один и тот же `inode` файла «`q.txt`». Ссылки на эти структуры помещаются в таблицу открытых процессом файлов под индексами 3 и 4, так как дескрипторы 0, 1 и 2 заняты стандартными потоками.

В первой итерации цикла системный вызов `write` для дескриптора `fd1` записывает символ «а» в открытый файл, что приводит к увеличению на 1 значения поля `f_pos` в соответствующей структуре `file`. Размер файла становится равным 1 байту.

Так как дескрипторам `fd1` и `fd2` соответствуют независимые структуры `file`, то для `fd2` значение поля `f_pos` остаётся равным 0. Поэтому на второй итерации цикла вызов `write` для `fd2` заменит символ «а» в файле на символ «б». Размер файла остаётся равным 1 байту, а значение `f_pos` для `fd2` увеличится на 1.

Поочерёдная запись символов в файл продолжается до конца цикла.

Таким образом, в файл будет записана только каждая вторая буква алфавита, начиная с «б», и итоговый размер файла составит 13 байт, так как вызов `write` для `fd2` будет заменять символы, записанные вызовом `write` для `fd1`.

4.3 Схема связи структур

Схема связи структур представлена на рисунке 3.4.

4.4 Вариант программы с `fopen`, `fprintf` и `fclose`

4.4.1 Код программы

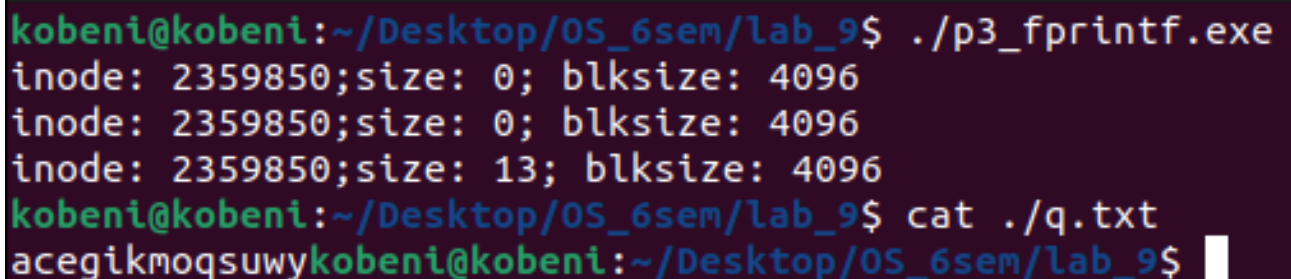
Листинг 4.2 — Код программы с `fopen`, `fprintf` и `fclose`

```
1 #include <fcntl.h>
2 #include <stdio.h>
3 #include <sys/stat.h>
4 #include <sys/types.h>
5
6 int main()
7 {
8     struct stat statbuf;
9     FILE *fs1 = fopen("q.txt", "w");
10    FILE *fs2 = fopen("q.txt", "w");
11    stat("q.txt", &statbuf);
12    printf("inode: %lu;size: %ld; blksize: %ld\n", statbuf.st_ino,
13          statbuf.st_size, statbuf.st_blksize);
14
15    for(char c = 'a'; c <= 'z'; c++)
16    {
17        if (c%2)
18            fprintf(fs1, "%c", c);
```


Продолжение листинга 4.2

```
18     else
19         fprintf(fs2, "%c", c);
20     }
21     stat("q.txt", &statbuf);
22     printf("inode: %lu;size: %ld; blksize: %ld\n", statbuf.st_ino,
           statbuf.st_size, statbuf.st_blksize);
23     fclose(fs2);
24     fclose(fs1);
25     stat("q.txt", &statbuf);
26     printf("inode: %lu;size: %ld; blksize: %ld\n", statbuf.st_ino,
           statbuf.st_size, statbuf.st_blksize);
27     return 0;
28 }
```

4.4.2 Результат работы программы



```
kobeni@kobeni:~/Desktop/OS_6sem/lab_9$ ./p3_fprintf.exe
inode: 2359850;size: 0; blksize: 4096
inode: 2359850;size: 0; blksize: 4096
inode: 2359850;size: 13; blksize: 4096
kobeni@kobeni:~/Desktop/OS_6sem/lab_9$ cat ./q.txt
acegikmoqsuwykobeni@kobeni:~/Desktop/OS_6sem/lab_9$
```

Рисунок 4.2 — Результат работы программы с `fopen`, `fprintf` и `fclose`

4.4.3 Анализ работы программы

Вызов библиотечной функции `fopen` откроет файл «q.txt» в режиме «только для записи» и вернёт два указателя на инициализированные структуры `FILE` (`fs1`, `fs2`). Во время вызова создаются две независимые структуры `file` с дескрипторами 3 и 4, значения которых будут записаны в поля `_fileno` соответствующих структур `_IO_FILE`. Эти структуры `file` ссылаются на один и тот же `inode` файла «q.txt».

Вызов библиотечной функции `fprintf` для `fs1` записывает символ «а» в буфер, что приводит к смещению в соответствующей структуре указателя `_IO_WRITE_PTR` на 1 байт.

Вызов библиотечной функции `fprintf` для `fs2` записывает символ «б» в буфер, что приводит к смещению в соответствующей структуре указателя `_IO_WRITE_PTR` на 1 байт.

В результате работы цикла в буфере `fs1` будет находиться каждая вторая буква алфавита, начиная с «а», а в `fs2` — каждая вторая буква алфавита, начиная с «б».

Вызов `fclose` для `fs2` приводит к записи из буфера в открытый файл. Значение поля `f_pos` для дескриптора 4 и размера файла становится 13. После этого дескриптор закрывается.

Так как были созданы две структуры открытых файлов, ссылающихся на один и тот же `inode`, а в структуре, соответствующей `fs1`, значение поля `f_pos` осталось равным 0, то при вызове `fclose` для `fs1` буфер записывается в открытый файл с начала, что приводит к замене последовательности символов буфера `fs2` на последовательность символов буфера `fs1`. Размер файла остаётся неизменным (13 байт).

Таким образом, итоговое содержимое файла зависит от порядка вызовов `fclose`. В данном случае в файл будет записан каждый второй символ алфавита, начиная с буквы «а», а размер файла составит 13 байт.

4.4.4 Схема связи структур

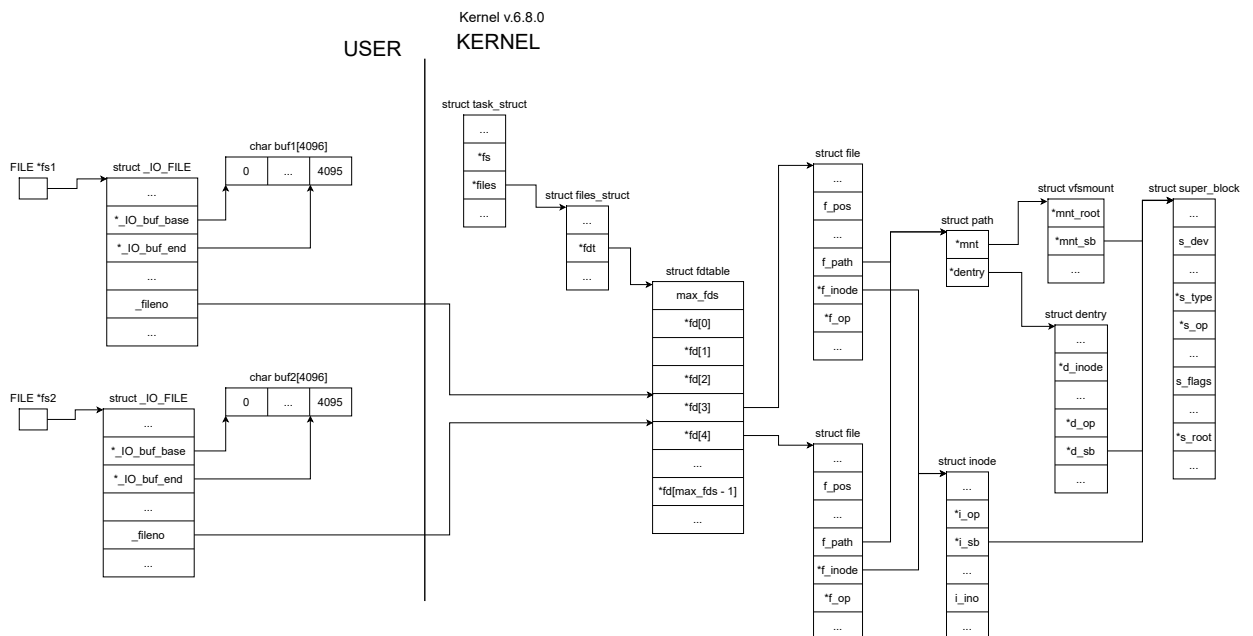


Рисунок 4.3 — Схема связи структур в третьей программе с `open`, `fprintf` и `fclose`